IT305 LAB: Socket Programming Project

Consolidated Design Document: Fault-Tolerant Topic-Based File Distribution Service

Merged Project Group: Group 103 & Group 106

September 27, 2026

Project Group Members & Contributions

Roll Number	Full Name	Primary Design Contribution
202401412	Yuvraj Chauhan(G-103)	Length-Prefixed Wire Framing & Message Formats (All Cases)
202401444	Digvijy Parmar(G-103)	Design Trade-offs, Extra Features, Instrumentation & Integration
202401476	Shlok Gohel(G-103)	Case 2 Session Management, Durability Invariant & Validation
202401496	Vaghela Khush(G-106)	Server Data Structures, CLI Parsing & Execution Loops
202401492	Vekariya Nisarg(G-106)	Case 2 Enhanced: Parallel TCP, `epoll`, `sendfile` & Bitmask Recovery
202401491	Vekariya Bhavya(G-106)	Chunk-Boundary Fault Injection & Case 1 Stateless Recovery

Contents

1 Server Data Structures and Directory Mapping **3**

1.1 Internal Data Structures & Topic Resolution . . . 3

1.2 Server Initialization, CLI Parsing & Execution Loops . . . 3

2 Client-Server Communication Protocol and Message Formats **3**

2.1 TCP Stream Framing and Common Header . . . 3

2.2 Explicit Payload Encoding and Binary Serialization Rules . . . 4

2.3 Unified Message Opcodes and Exact Binary Payload Layouts . . . 4

2.4 Case-by-Case Protocol Breakdown . . . 5

3 Fault Injection Mechanism and Part II Case 1 Recovery **5**

3.1 Chunk-Boundary Fault Injection Model . . . 5

3.2 Part II Case 1: Stateless Recovery and Bounded Retry Policy . . . 6

4 Session Management Protocol (Part II - Case 2) **6**

4.1 Session Abstraction and Standardized 128-Bit Session Identifier . . . 6

4.2 State Ownership, Checkpoint Frequency, and Durability Execution Order . . . 7

4.3 Completed-Topic Boundary Handling & Seven-Step Server Validation . . . 7

5 Planned Performance Enhancements (Part II - Case 2 Enhanced) **9**

5.1 Non-Blocking `epoll` Event Loop & Zero-Copy `sendfile(2)` . . . 9

5.2 Parallel TCP Connections (25% Single-Stream Capacity Impact) & Scheduling . . . 9

5.3 Sub-Stream Fault Isolation & Chunk-Bitmask Recovery . . . 9

6 Extra Properties and Features Beyond Minimum Requirements **9**

7 Code Architecture and Module Structure **11**

8 Design Thought Process and Alternatives Considered **12**

9 Performance Instrumentation Plan and Analytical Projections **13**

9.1 Experimental Setup and Metrics Definition 13

9.1.1 Part I Client Concurrency Plan and Aggregate Throughput Formula 13

9.2 Analytical Performance Projections (Target Baselines) 13

1 Server Data Structures and Directory Mapping

1.1 Internal Data Structures & Topic Resolution

The server utilizes an in-memory Hash Map (`std::unordered_map<std::string, TopicEntry>` in C++ or a chained Hash Table in C) as the primary internal data structure to map topic strings to their resolved directory paths and cached file lists. A Hash Map is selected over a prefix tree because clients request exact topic string matches (e.g., `"Cane"`, `"Cavallo"`, `"Elefante"`), making the $O(1)$ average time complexity of a hash lookup the most efficient approach for resolving `GET <topic>` requests. Because the dataset's directory structure is static during runtime, this Hash Map is populated entirely at server startup, eliminating the need for read/write concurrency locks during concurrent client processing.

To populate this data structure dynamically rather than hard-coding paths, a directory-scanning routine executes during server initialization using POSIX `<dirent.h>` (`opendir()` and `readdir()`). For every valid sub-folder found within `<topics_root_dir>`, the routine extracts the folder name as the topic key, constructs the canonical path (e.g., `<topics_root_dir>/Cane`), caches a deterministically sorted vector of regular files (and their byte sizes via `stat()`) inside that topic directory, and inserts the key-value entry into the Hash Map.

1.2 Server Initialization, CLI Parsing & Execution Loops

To comply with automated grading scripts, the CLI parsing routine strictly enforces the required invocation formats without interactive prompts:

- **Part I Server:** `./server <port> <topics_root_dir>` (verifies `argc >= 3`, validates user port range 1024–65535, and checks directory existence via `stat()`).
- **Part II Server (All Cases):** `./server <port> <topics_root_dir> <failure_probability>` (additionally validates real-valued failure probability $P_f \in [0.0, 1.0]$).
- **Client (All Cases):** `./client <server_ip> <server_port> <topic> <output_dir>` (creates `<output_dir>/<topic>/` and writes incoming topic files).

Sequential vs. Multi-Threaded Execution Loops:

1. **Sequential Request Loop (Part I Single-Threaded):** Operates as an infinite `while(running)` loop around `accept()`. When a client connects, the main thread passes the descriptor directly to `process_client()` and blocks until all files for the requested topic are transmitted and the socket closes.
2. **Multi-Threaded Request Loop (Part I Multi-Threaded & Part II):** The main thread remains dedicated to `accept()`. Upon each connection, it allocates a thread-argument context containing the client socket descriptor and invokes `pthread_create()` to spawn a detached POSIX worker thread executing `process_client()` independently.

2 Client-Server Communication Protocol and Message Formats

2.1 TCP Stream Framing and Common Header

TCP preserves byte order but does not preserve application message boundaries: a single `send()` may split across multiple `recv()` calls, or multiple small messages may coalesce. Furthermore, image files contain arbitrary binary data, making delimiter-based framing unsafe for payloads.

Chosen Framing Solution: Length-prefixed binary frames supported by loop-complete socket helpers (`send_all()`, `recv_all()`, `send_frame()`, `recv_frame()`). Every message—whether control metadata or binary file data—begins with a fixed 8-byte binary header in network byte order (`big-endian`), followed by a variable-length binary payload of exactly `Payload Length` bytes:

Header Field	Binary Type	Purpose
`Magic`	`uint16_t` (2 B)	Protocol identifier (`0x4954`, "IT") to detect framing desynchronization.
`Version`	`uint8_t` (1 B)	Protocol revision (`0x01`).
`Type`	`uint8_t` (1 B)	Message opcode (`0x01`–`0x0E`) distinguishing control frames from data frames.
`Payload Length`	`uint32_t` (4 B)	Unsigned 32-bit byte length of the payload immediately following the 8-byte header.
`Payload`	`uint8_t[]` (Var)	Deterministically serialized binary control fields or raw binary file chunk data.

Table 1: Common 8-byte length-prefixed frame header used across all project cases.

2.2 Explicit Payload Encoding and Binary Serialization Rules

To eliminate any ambiguity in message encoding and avoid compiler-dependent struct padding, all frame payloads use explicit binary serialization:

1. **Fixed-Width Binary Integers:** Every integer field in a message payload is serialized as an unpadded, fixed-width unsigned binary integer (`uint16_t` = 2 bytes, `uint32_t` = 4 bytes, `uint64_t` = 8 bytes) encoded in **network byte order (big-endian)** via `htons()`, `htonl()`, and `htobe64()`.

2. **128-Bit Session Identifier (`uint8_t[16]`):** The session identifier is strictly a **128-bit (16-byte) value** throughout the entire system. On the wire, it is transmitted as a fixed 16-byte binary array (`uint8_t[16]`). In stateless modes (Part I and Part II Case 1), all 16 bytes are set to zero (`0x00...00`). Only when written to the human-readable client checkpoint file or diagnostic logs is this 128-bit value rendered as a 32-character lowercase hexadecimal ASCII string (`char[33]` including the null terminator).

3. **Length-Prefixed Variable Strings:** All variable-length strings (`topic`, `filename`, `reason`) are binary-encoded as a 2-byte unsigned byte length (`uint16_t len` in network byte order) immediately followed by `len` raw UTF-8 bytes without a null terminator. This allows filenames to contain spaces or arbitrary characters without delimiter collisions.

4. **Control vs. Data Distinction & Frame Validation:** The receiver distinguishes control messages from file-chunk payloads strictly via the header's `Type` byte (`0x05 FILE_DATA` carries file chunk bytes; all other opcodes carry control metadata). Any header with `Magic != 0x4954`, `Version != 0x01`, or `Payload Length > MAX_FRAME_PAYLOAD` (1 MB) is rejected immediately with `PROTOCOL_ERROR`.

2.3 Unified Message Opcodes and Exact Binary Payload Layouts

Table 2 specifies the exact byte-by-byte binary layout for every opcode across Part I and Part II.

Code	Message Name	Dir.	Exact Binary Payload Layout (Big-Endian) & Purpose
0x01	GET	$C \rightarrow S$	[topic_len:uint16_t][topic:uint8_t[topic_len]]: Requests all files belonging to topic.
0x02	GET_OK (OK)	$S \rightarrow C$	[session_id:uint8_t[16]][file_count:uint32_t][total_bytes:uint64_t]: Accepts request (session_id is 16 zero bytes in Part I and Case 1).
0x03	ERROR (ERR)	$S \rightarrow C$	[err_code:uint16_t][msg_len:uint16_t][reason:uint8_t[msg_len]]: Reports 404 TOPIC_NOT_FOUND, 400 BAD_REQUEST, or 500 IO_ERROR.
0x04	FILE_BEGIN (FILE)	$S \rightarrow C$	[file_index:uint32_t][byte_offset:uint64_t] [bytes_following:uint64_t][name_len:uint16_t][filename:uint8_t[name_len]]: Announces file metadata and remaining byte count to transfer.
0x05	FILE_DATA	$S \rightarrow C$	[file_index:uint32_t][offset:uint64_t][data_len:uint32_t] [data:uint8_t[data_len]]: Transfers one binary chunk of file data ($\leq$ 64 KB).
0x06	FILE_END	$S \rightarrow C$	[file_index:uint32_t]: Marks completion of the current file.
0x07	TRANSFER_END (DONE)	$S \rightarrow C$	[files_sent:uint32_t][useful_bytes:uint64_t]: Marks complete topic delivery; verified by client before deleting checkpoint.
0x08	CHECKPOINT_ACK	$C \rightarrow S$	[session_id:uint8_t[16]][file_index:uint32_t][ack_offset:uint64_t]: Optional client telemetry confirming durable disk flush.
0x09	RESUME	$C \rightarrow S$	[session_id:uint8_t[16]][file_index:uint32_t][byte_offset:uint64_t] [topic_len:uint16_t][topic:bytes][name_len:uint16_t][filename:bytes]: Reports client's last durable checkpoint on reconnect.
0x0A	RESUME_OK (CONT)	$S \rightarrow C$	[file_index:uint32_t][accepted_offset:uint64_t][rem_files:uint32_t] [rem_bytes:uint64_t]: Confirms validated resume position.
0x0B	RESTART	$S \rightarrow C$	[code:uint16_t][msg_len:uint16_t][reason:uint8_t[msg_len]]: Rejects invalid/expired resume (410 EXPIRED, 409 MISMATCH, 416 INVALID_OFFSET); triggers client fallback to fresh GET.
0x0C	GET_ENHANCED	$C \rightarrow S$	[num_streams:uint16_t][topic_len:uint16_t][topic:uint8_t[topic_len]]: Initiates a parallel multi-connection transfer session.
0x0D	JOIN_SESSION	$C \rightarrow S$	[session_id:uint8_t[16]][conn_id:uint32_t][bmap_len:uint32_t] [chunk_bitmap:uint8_t[bmap_len]]: Attaches parallel data socket to session.
0x0E	ASSIGN_WORK	$S \rightarrow C$	[conn_id:uint32_t][start_unit:uint32_t][end_unit:uint32_t]: Dispatches file or 64 KiB chunk range to a parallel stream.

Table 2: Unified protocol message opcodes and exact binary payload definitions.

2.4 Case-by-Case Protocol Breakdown

- **Part I (Single & Multi-Threaded):** Uses opcodes 0x01–0x07 with a zeroed 16-byte session_id. Sequence: GET $\rightarrow$ GET_OK $\rightarrow$ (FILE_BEGIN $\rightarrow$ FILE_DATA$\times k$ $\rightarrow$ FILE_END) per file $\rightarrow$ TRANSFER_END.
- **Part II Case 1 (No Session Management):** Uses only opcodes 0x01–0x07 (session_id = 0). On any mid-transfer connection failure, the client reconnects and sends a fresh GET (0x01), restarting the topic from File 0, Byte 0.
- **Part II Case 2 (With Session Management):** Uses opcodes 0x01–0x0B. GET_OK returns a random 128-bit session_id. On reconnect after a fault, the client sends RESUME (0x09); the server replies RESUME_OK (0x0A) to resume from accepted_offset, or RESTART (0x0B) to trigger a clean fallback to GET.
- **Part II Case 2 Enhanced (Parallel TCP):** Uses opcodes 0x01–0x0E. The primary connection sends GET_ENHANCED (0x0C), while N parallel sockets attach via JOIN_SESSION (0x0D) and receive work ranges via ASSIGN_WORK (0x0E).

3 Fault Injection Mechanism and Part II Case 1 Recovery

3.1 Chunk-Boundary Fault Injection Model

In Part II, the multi-threaded server accepts a failure probability $P_f \in [0.0, 1.0]$ via the third CLI argument. Rather than dropping connections before any payload flows, the fault decision is evaluated **once at the**

boundary of each transmitted chunk (immediately after `send_frame()` completes for a 64 KB `FILE_DATA` chunk):

1. **Per-Thread PRNG State & Independence:** Each server worker thread instantiates an independent, thread-local Mersenne Twister engine (`std::mt19937`) seeded via `std::random_device` combined with the thread ID and monotonic clock. This avoids mutex contention on global PRNG state and guarantees that fault decisions across chunks and across concurrent client threads are statistically independent.
2. **Decision Rule:** After sending a chunk, the worker draws $r \sim \text{Uniform}(0.0, 1.0)$. If $r < P_f$, the server logs a `[FAULT]` event recording the topic, current file, and bytes sent so far, immediately calls `close(client_socket)`, and exits the worker thread.
3. **Cumulative Topic Failure Probability:** Under the independent per-chunk decision model, a topic spanning n chunks experiences at least one interruption with probability:

$$P(\text{at least one fault in topic}) = 1 - (1 - P_f)^n$$

   Evaluating faults strictly post-chunk guarantees that at least one chunk of real data is transferred before a drop occurs, forcing recovery paths to handle genuine partial files. For completion benchmarks, P_f is evaluated over $[0.0, 1.0)$ (e.g., $0.0, 0.1, 0.3, 0.5$); setting $P_f = 1.0$ deterministically closes the socket after the first chunk of every attempt, preventing stateless Case 1 from advancing past Chunk 1.

3.2 Part II Case 1: Stateless Recovery and Bounded Retry Policy

Case 1 establishes the fault-tolerant baseline where **no metadata about an in-progress transfer is retained by either client or server**:

- **Failure Detection:** The client detects an unexpected server drop when `recv_all()` returns `0` (premature EOF before `TRANSFER_END`) or `-1` with `ECONNRESET`/`EPIPE`.
- **Bounded Reconnect Policy with Backoff:** To prevent infinite livelock under extreme fault rates (such as $P_f = 1.0$), the client enforces a configurable retry limit (`MAX_RETRIES = 50` by default, adjustable via `--max-retries`) with truncated exponential backoff (50 ms initial delay, doubling up to 1000 ms). On each retry, the client closes its broken socket, reconnects, and sends an identical `GET <topic>` request. If `MAX_RETRIES` is reached without receiving `TRANSFER_END`, the client aborts with a non-zero exit code and logs `status = INCOMPLETE`.
- **File Overwrite Semantics:** The server restarts streaming from File 0, Byte 0 on every Case 1 retry. The client opens target files with `O_WRONLY | O_CREAT | O_TRUNC`, overwriting any previously written files in `<output_dir>/<topic>/`. All bytes received prior to the failure are logged as redundant bytes.

4 Session Management Protocol (Part II - Case 2)

4.1 Session Abstraction and Standardized 128-Bit Session Identifier

In Case 2, a **session** represents the logical transfer of one topic to one client and outlives individual TCP connections. Upon receiving an initial `GET <topic>`, the server generates an unguessable **128-bit session identifier** (16 random bytes read from `/dev/urandom`) and returns it in the `GET_OK` frame as `uint8_t[16]`. Using an explicit 128-bit `session_id` is mandatory because reconnecting clients arrive on new ephemeral TCP ports, and multiple concurrent clients on the same testbed node may request the exact same topic simultaneously.

Checkpoint Field	Representation (Wire / Disk)	Purpose in Recovery Protocol
`session_id`	128-bit ID: `uint8_t[16]` on wire; `char[33]` hex on disk	Uniquely identifies the active transfer session in the server's session hash table upon reconnect.
`topic`	`char[64]` (length-prefixed on wire)	Prevents a checkpoint from being replayed against a different topic.
`file_index`	`uint32_t` (0 . . . file_count)	Index into the server's sorted topic file list; equals `file_count` when all files are completed.
`file_name`	`char[256]` (length-prefixed on wire)	Verifies dataset consistency (maps to file at `file_index`, or `"EOF"` when file_index == file_count).
`byte_offset`	`uint64_t`	Exact number of bytes durably flushed (`fsync`) to disk for the current file.
`files_done`	`uint32_t`	Count of completely transferred files (used for final completion verification).

Table 3: Structure of the persistent client checkpoint record.

4.2 State Ownership, Checkpoint Frequency, and Durability Execution Order

Dual-Side State Roles: The client's on-disk checkpoint is **authoritative** for recovery progress, while the server's in-memory session table (`session_id[16]`, `topic`, `file_index`, `bytes_sent_highwater`, `last_active`, protected by a `pthread_mutex_t`) serves as a **validator**. Relying on the server's `send()` byte count alone would be unsafe because `send()` only copies bytes into the kernel socket buffer—if the connection drops before those bytes reach the client, resuming from the server's counter would leave an undetected gap in the client's file.

When `write()` vs. `fsync()` Executes ($W = 64$ KB Window): Calling `fsync()` on every small socket read would severely degrade disk throughput. Instead, the client separates normal chunk writes from periodic checkpoint commits:

1. **Buffered Chunk Writes:** As `FILE_DATA` frames arrive, the client writes the payload bytes at the specified file offset using standard buffered `write()` calls without invoking `fsync()` on sub-window reads.
2. **Checkpoint Trigger Condition:** A durable checkpoint commit executes **only** when (i) accumulated bytes written since the last checkpoint reach the configured checkpoint window $W = 64$ KB (adjustable via `--ckpt-window`), or (ii) a file finishes (`FILE_END` received).
3. **Strict Commit Sequence:** Whenever a checkpoint is triggered, the client executes three steps in strict order:
   (a) Call `fflush()` and `fsync()` on the active target file descriptor in `<output_dir>/<topic>/<file_name>` so all payload bytes up to the new `byte_offset` are physically durable on disk.
   (b) Write the single-line ASCII checkpoint record (`<session_id_hex32> <topic> <file_index> <byte_offset> <files_done> <file_name>\n`) to `<output_dir>/.it305_<topic>.ckpt.tmp`, call `fsync()` on the temporary descriptor, and atomically `rename()` it over `<output_dir>/.it305_<topic>.ckpt`.
   (c) Optionally send a non-blocking `CHECKPOINT_ACK` (`0x08`) frame for server progress logging (recovery never depends on `CHECKPOINT_ACK`; the client's `RESUME` message carries the full checkpoint state).

Consistency Invariant: *The durable checkpoint offset may lag the data written to disk by at most $W = 64$ KB, but it can never lead it.* Upon reconnection, the client opens the target file with `open(path, O_WRONLY)` and seeks via `lseek(fd, accepted_offset, SEEK_SET)` (never `O_TRUNC` or `O_APPEND`). Any uncheckpointed bytes written beyond `accepted_offset` prior to the crash ($\leq W$ bytes) are cleanly overwritten in place.

4.3 Completed-Topic Boundary Handling & Seven-Step Server Validation

File-Boundary & Completed-Topic Normalization: To ensure completed files and completed topics are handled deterministically if a connection drops right before `TRANSFER_END` arrives, the client normalizes its

checkpoint immediately upon receiving `FILE_END` for file k:

- **Mid-Topic File Completion ($k+1 <$ file_count):** The checkpoint advances to the beginning of the next file: `file_index` $= k+1$, `byte_offset` $= 0$, `files_done` $= k+1$, and `file_name` = name of file $(k+1)$.
- **End-of-Topic Completion ($k+1 ==$ file_count):** When the final file is durably flushed to disk, the checkpoint records the explicit terminal state: `file_index` = file_count, `byte_offset` $= 0$, `files_done` = file_count, and `file_name` = `"EOF"`.

Seven-Step Validation Logic (Including `file_index == file_count`): When a reconnecting client sends `RESUME`, the server locks the session table, runs a lazy TTL sweep (600s idle limit), and evaluates seven checks:

1. 128-bit `session_id` exists in the active session table (otherwise reply `RESTART 410 SESSION_EXPIRED`).
2. Session has not exceeded the 600-second idle TTL (otherwise `RESTART 410`).
3. Requested `topic` matches the session's recorded topic (otherwise `RESTART 409 TOPIC_MISMATCH`).
4. **Index Range Check ($0 \leq$ `file_index` $\leq$ `file_count`):**
   - *Terminal Case (`file_index` == `file_count`):* Valid if and only if `byte_offset` $== 0$, `file_name` == `"EOF"`, and the server has already sent all files. The server immediately replies `RESUME_OK` (with `rem_files` $= 0$, `rem_bytes` $= 0$) followed by `TRANSFER_END` without re-sending any file data.
   - *Active File Case ($0 \leq$ `file_index` $<$ `file_count`):* Proceeds to Checks 5–7. Any `file_index` $>$ `file_count` is rejected with `RESTART 416 OFFSET_INVALID`.
5. Server's filename at `file_index` matches `file_name` (otherwise `RESTART 409`).
6. $0 \leq$ `byte_offset` $\leq$ `file_size` of that file (otherwise `RESTART 416`).
7. Cumulative bytes claimed ($\leq$ `bytes_sent_highwater`) do not exceed what the server actually sent (otherwise `RESTART 416`).

If all checks pass for an active file, the server replies with `RESUME_OK`, seeks via `fseeko(fp, byte_offset, SEEK_SET)`, and sends `FILE_BEGIN` with `bytes_following` = `file_size` − `byte_offset`. On `RESTART`, the client removes its stale `.ckpt` file and falls back to a fresh `GET <topic>` (bounded by `MAX_RETRIES`). Upon verifying `TRANSFER_END`, the client unlinks `.ckpt`.

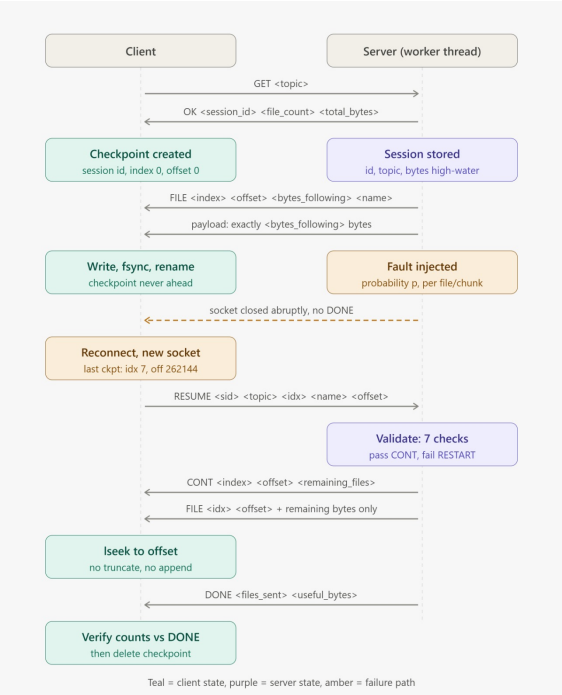


Figure 1: Case 2 Session Management, Checkpoint Durability, and 7-Check Validation Flow.

5 Planned Performance Enhancements (Part II - Case 2 Enhanced)

Case 2 Enhanced combines socket-level I/O optimizations with parallel TCP connections to maximize throughput and minimize recovery overhead under high fault rates:

5.1 Non-Blocking `epoll` Event Loop & Zero-Copy `sendfile(2)`

To reduce context-switching and memory-copy overhead under high concurrency, Case 2 Enhanced upgrades the server I/O pipeline:

- **Non-Blocking Sockets with `epoll`:** Socket descriptors are set to `O_NONBLOCK` and monitored via Linux `epoll`. Unlike `select()`/`poll()`, which require an $O(N)$ linear scan over all N registered descriptors on every call, `epoll_wait()` scales as $O(K)$ in the number of *ready* (active) events—yielding amortized $O(1)$ event notification per active socket when concurrency is high and only a subset of sockets are ready simultaneously—while descriptor insertion, modification, and removal via `epoll_ctl()` operate in $O(\log N)$ time against the kernel's internal red-black tree.
- **Zero-Copy Disk-to-Socket Transfer:** Rather than copying file blocks into user space via `read()` and back into kernel space via `send()`, worker threads call `sendfile(out_socket, in_file_fd, &offset, count)` to transfer payload bytes directly from the kernel page cache to the NIC DMA ring buffer.

5.2 Parallel TCP Connections (25% Single-Stream Capacity Impact) & Scheduling

A single TCP stream loses 50% of its congestion window (`cwnd`) on a single packet-loss event under AIMD and loses 100% of its transfer capacity during a connection drop. By opening $N = 4$ simultaneous TCP connections per topic (configurable via optional trailing CLI argument, defaulting to $N = 4$), **under a simplified equal-bandwidth assumption across the 4 streams, a single-stream interruption temporarily removes $\frac{1}{4} \times 100\% = 25\%$ of the aggregate connection capacity** rather than stalling the entire transfer, while the remaining 3 streams continue transferring concurrently. (In practice, exact application-level throughput impact depends on per-stream congestion windows, work queue scheduling, network RTT, and reconnection latency.)

After the primary connection sends `GET_ENHANCED` and receives the 128-bit `session_id`, the client opens $N = 4$ parallel data sockets, each sending `JOIN_SESSION(session_id, conn_id)`. Work is dispatched via `ASSIGN_WORK` using two supported granularities:

1. **File-Range Partitioning:** Disjoint file index ranges (e.g., `C0: 0-24, C1: 25-49`) are assigned per stream so each connection writes independent files.
2. **Dynamic 64 KiB Chunk Work-Queue with `pwrite(2)`:** To prevent stragglers when file sizes vary, files are divided into 64 KiB work units pulled dynamically from a thread-safe server queue. The client pre-allocates target files via `posix_fallocate()` and writes incoming chunks concurrently using positional `pwrite(fd, buf, len, offset)` without user-space mutex contention.

5.3 Sub-Stream Fault Isolation & Chunk-Bitmask Recovery

When the server's fault injector terminates one parallel stream (with probability P_f), surviving streams continue unaffected. The server tracks a 3-state lifecycle for every 64 KiB chunk (`UNASSIGNED` → `IN_FLIGHT` → `COMMITTED`). When a sub-stream drops, only its `IN_FLIGHT` chunk is atomically requeued. The client replaces the dropped socket, sends `JOIN_SESSION` with its `chunk_bitmap`, and receives only missing 64 KiB blocks.

6 Extra Properties and Features Beyond Minimum Requirements

Beyond the baseline specification, our design incorporates four production-grade features:

- **Dynamic Dataset Extensibility:** By scanning `<topics_root_dir>` dynamically via `dirent.h` at startup rather than hard-coding the 10 Animals-10 folder names, new topic folders can be added and served immediately without recompiling.

- **Graceful Shutdown via POSIX Signals:** Catching `SIGINT` and `SIGTERM` halts the `accept()` loop cleanly, waits for active worker threads to flush current checkpoints, closes descriptors, and frees heap allocations to guarantee zero memory leaks under Valgrind.
- **Non-Interactive Background Logging:** To ensure automated grading scripts never hang on console prompts, all status monitoring (`[INFO]`, `[FAULT]`, `[RESUME]`) and CSV metric telemetry are written asynchronously via a mutex-guarded logger to `Results/*.csv` and log files.
- **Backwards-Compatible CLI Extensibility:** Optional trailing flags (`--ckpt-window`, `--streams`, `--max-retries`, `--seed`) allow fine-grained benchmarking control while strictly preserving the exact positional CLI invocation required by the auto-grader.

7 Code Architecture and Module Structure

To satisfy the final submission layout, the repository is organized into four independent build directories (`Part1/`, `Part2_Case1/`, `Part2_Case2/`, `Part2_Case2_Enhanced/`) plus `Results/`. Running `make` inside any case directory compiles and links the modules into exactly two executables: `server` and `client`.

Module / File	**Key Functions**	**Responsibility Across Cases**
`server.c/cpp`	`main(argc, argv)`	Validates CLI args, initializes topic map, registers signal handlers, and launches the sequential or multi-threaded accept loop.
`client.c/cpp`	`main()`, `request_topic()`, `reconnect_loop()`	Parses CLI args, manages bounded connection retries with backoff, coordinates checkpoint loads, and verifies `TRANSFER_END` counters.
`directory_mapper.c/h`	`scan_and_map_topics()`, `get_topic_path()`	Traverses `<topics_root_dir>` at startup and provides lock-free $O(1)$ hash map lookups for worker threads.
`protocol.c/h`	`send_all()`, `recv_all()`, `send_frame()`, `recv_frame()`	Encapsulates the 8-byte length-prefixed framing layer and fixed-width big-endian binary serializers for opcodes `0x01`–`0x0E`.
`concurrency.c/h`	`run_sequential_loop()`, `run_concurrent_loop()`, `worker_thread()`	Implements single-threaded blocking execution and detached `pthread_create()` worker dispatch (plus `epoll` pool in Enhanced).
`fault.c/h`	`init_thread_rng()`, `should_inject_fault(pf)`	Implements independent thread-local `mt19937` uniform draw evaluated at chunk boundaries in Part II.
`session.c/h` & `checkpoint.c/h`	`create_session()`, `validate_checkpoint()`, `save_checkpoint()`	Manages 128-bit server session state, 7-step boundary-aware validation (file_index $\leq$ file_count), and client atomic `fsync` + `rename`.
`metrics_logger.c/h`	`record_transfer_metrics()`, `log_event()`	Captures `clock_gettime(CLOCK_MONOTONIC)` durations and byte counters, appending rows to `Results/*.csv`.

Table 4: Modular file structure and function responsibilities.

8 Design Thought Process and Alternatives Considered

Design Decision	Alternatives Considered	Chosen Approach & Rationale
Topic Lookup Structure	1. Filesystem traversal per request. 2. Prefix tree (Trie). 3. Static startup Hash Map.	**Startup Hash Map:** Exact topic string matching runs in $O(1)$ average time and remains read-only after initialization, requiring zero lock contention across threads.
Message Framing & Serialization	1. Delimiter-based text streams. 2. Compiler-packed C structs. 3. 8-byte binary frame + big-endian length-prefixed fields.	**Length-Prefixed Framing & Explicit Binary Encoding:** JPEG images contain arbitrary binary bytes; explicit big-endian integer widths and string lengths avoid struct-padding bugs.
Fault Injection Point	1. Connection-accept time drop. 2. Timer-driven asynchronous signal. 3. Post-chunk boundary draw.	**Post-Chunk Boundary + Thread-Local RNG:** Guarantees data has already flowed before failure, deterministically testing partial-file recovery without cross-thread RNG locks.
Session Identification	1. Client `(IP, port)` tuple. 2. 64-bit counter. 3. Server-issued 128-bit `session_id`.	**128-Bit `session_id` (`uint8_t[16]` / 32-hex):** Standardized single 128-bit representation prevents port-change collisions and concurrent same-topic ambiguity.
Checkpoint Commit & `fsync` Cadence	1. `fsync()` after every socket read. 2. Per-file checkpoint only. 3. Buffered writes + `fsync()` every $W = 64$ KB and per file.	**Windowed Commit ($W = 64$ KB):** Avoids disk I/O stalls on small reads while bounding crash retransmission to at most 64 KB via atomic temp-file `rename()`.
Client Resume File I/O	1. `O_TRUNC` (rewrite). 2. `O_APPEND`. 3. `O_WRONLY` + `lseek()`.	**Positioned Overwrite (`lseek`):** `O_APPEND` duplicates lagging bytes and corrupts files; positioned `lseek` overwrites identical bytes safely if a checkpoint lags by $\leq W$.
Enhanced Parallelism & I/O Multiplexing	1. Single blocking socket. 2. `select()`/`poll()` $O(N)$ scan. 3. 4× Parallel TCP + `epoll` ($O(K)$ ready events, $O(\log N)$ ctl) + `sendfile`.	**4× Parallel TCP + Dynamic 64 KiB Chunks:** Under an equal-bandwidth assumption, losing 1 of 4 streams temporarily removes 25% capacity rather than 100%; `pwrite(2)` enables lock-free client writes.

Table 5: Summary of architectural alternatives considered and justifications.

9 Performance Instrumentation Plan and Analytical Projections

9.1 Experimental Setup and Metrics Definition

Experiments will execute across three networked lab computers (Node 1 running the server; Nodes 2 and 3 spawning concurrent clients). High-resolution timing is captured via `clock_gettime(CLOCK_MONOTONIC)` from `<time.h>`. For each individual client transfer, the CSV logger records:

- **Useful Bytes (B_{useful}):** Exact byte size of the final verified files written to `<output_dir>/<topic>/`.
- **Total Bytes (B_{total}):** All bytes transmitted over the socket across all attempts, including 8-byte frame headers, control payloads, and retransmitted chunks.
- **Redundant Bytes ($B_{\text{redundant}}$):** Payload bytes retransmitted due to connection drops ($B_{\text{payload_sent}} - B_{\text{useful}}$).
- **Single-Client Goodput, Wire Throughput & Overhead:**

$$\text{Goodput} = \frac{B_{\text{useful}}}{T_{\text{complete}}}, \qquad \text{Wire Throughput} = \frac{B_{\text{total}}}{T_{\text{complete}}}, \qquad \text{Overhead (\%)} = \frac{B_{\text{total}} - B_{\text{useful}}}{B_{\text{useful}}} \times 100$$

9.1.1 Part I Client Concurrency Plan and Aggregate Throughput Formula

To satisfy the Part I requirement of measuring aggregate file transfer rate versus the number of clients for single-threaded and multi-threaded servers, experiments will explicitly test client concurrency counts of:

$$N \in \{1, 2, 4, 8, 16, 32\} \text{ concurrent clients}$$

distributed evenly across the two client machines (Nodes 2 and 3) while keeping the server node (Node 1), dataset, and requested topics consistent across runs. Each client count N will be evaluated separately on the single-threaded server and the multi-threaded server across multiple trials. For a concurrent run of N clients, **Aggregate File Transfer Rate (Aggregate Goodput)** is defined as:

$$G_{\text{aggregate}}(N) = \frac{\sum_{i=1}^{N} B_{\text{useful},i}}{T_{\text{experiment}}}$$

where $B_{\text{useful},i}$ is the useful payload bytes delivered to client i, and $T_{\text{experiment}} = \max_i(t_{\text{end},i}) - \min_i(t_{\text{start},i})$ is the elapsed wall-clock time from when the first client starts its request until all N clients complete their transfers. Results will be plotted as curves of $G_{\text{aggregate}}(N)$ versus N comparing the single-threaded and multi-threaded implementations.

9.2 Analytical Performance Projections (Target Baselines)

Table 6 specifies the projected Part I aggregate transfer rate ($G_{\text{aggregate}}$ vs. client count N) for single-threaded and multi-threaded servers, and Table 7 specifies the Part II fault-tolerance projections across failure probabilities $P_f \in \{0.0, 0.1, 0.3, 0.5\}$ at $N = 8$ concurrent clients:

Part I Server Mode	$N = 1$	$N = 2$	$N = 4$	$N = 8$	$N = 16$	$N = 32$	**Expected Scaling vs. N**
Single-Threaded	~ 22 MB/s	~ 22 MB/s	~ 21 MB/s	~ 21 MB/s	~ 20 MB/s	~ 20 MB/s	Flat aggregate rate; clients wait sequentially in `accept()` queue.
Multi-Threaded	~ 22 MB/s	~ 41 MB/s	~ 64 MB/s	~ 82 MB/s	~ 88 MB/s	~ 87 MB/s	Scales with N until saturating NIC and disk I/O bandwidth.

Table 6: Part I Analytical Target Projections: Aggregate File Transfer Rate $G_{\text{aggregate}}(N)$ vs. Number of Clients N.

Part II Scenario ($N = 8$)	**Projected Goodput** ($P_f = 0.0$)	**Projected Goodput** ($P_f = 0.3$)	**Expected Retransmission Overhead & Recovery Behavior**
Case 1 (No Session)	$\sim$ 80–85 MB/s	$\sim$ 6–10 MB/s	Severe degradation ($>$ 300% redundant byte overhead) due to full topic restarts from Byte 0 on every fault.
Case 2 (Checkpoint)	$\sim$ 78–83 MB/s	$\sim$ 42–48 MB/s	Redundant bytes per fault bounded by checkpoint window $W = 64$ KB ($<$ 8% overhead).
Case 2 Enhanced (4× TCP)	$\sim$ 105–115 MB/s	$\sim$ 80–86 MB/s	`sendfile` zero-copy + 4× parallel TCP (25% capacity loss on single-stream fault under equal-bandwidth assumption) + 64 KiB bitmask recovery ($<$ 3% overhead).

Table 7: Part II Analytical Target Projections: Goodput and Retransmission Overhead under Fault Injection.